

Walkthrough - Student Performance Analytics Portal (Planning & Initial Setup)

We have completed the project planning and initial setup phase for the **Student Performance Analytics Portal** as outlined in the task requirements.

Deliverables Created

1. MERN Project Skeleton

- Root configuration and workspaces setup ([package.json](#), [.gitignore](#)).
- Complete `backend/` directory structure with Express server ([server.js](#)), database configuration ([db.js](#)), authentication middleware ([authMiddleware.js](#)), entity routing shells, and Mongoose schemas for [User](#), [Student](#), [Course](#), [Grade](#), and [Attendance](#).
- Scaffolded `frontend/` directory configured with Vite + React + Vanilla CSS.

2. Interactive Planning & Prototype Dashboard

Instead of static mockup pictures, we built a fully interactive dashboard web application.

- **Login screen:** Seamless mock authentication switching between roles.
- **Admin Dashboard:** Visualizes student registration counts, staff stats, database status, and includes an interactive user creation form.
- **Teacher Dashboard:** Displays class GPAs, averages, and features an interactive gradebook where editing marks recalculates class aggregates in real-time.
- **Student Dashboard:** Showcases GPAs, attendance charts, teacher announcements, and a dynamic custom SVG trend-line tracing academic growth.
- **Specs & Documentation Hub:** Hosts direct markdown displays for the Architecture Diagram, Database Schemas, API Route Specs, and Permissions Matrix. Includes a Swagger-style API simulator demonstrating mock response outputs on button triggers.

3. Core Planning Sheets

- [architecture_diagram.md](#) - Details client-server communication with a Mermaid graph.
- [database_schema.md](#) - Details MongoDB collections and relations.
- [api_documentation.md](#) - Lists REST endpoints, authorization scopes, and JSON models.
- [user_roles.md](#) - Establishes authorization matrix for Admin, Teacher, Student roles.
- [wireframes.md](#) - Specifications sheet outlining custom Vanilla CSS design tokens.

4. Git Repository Setup

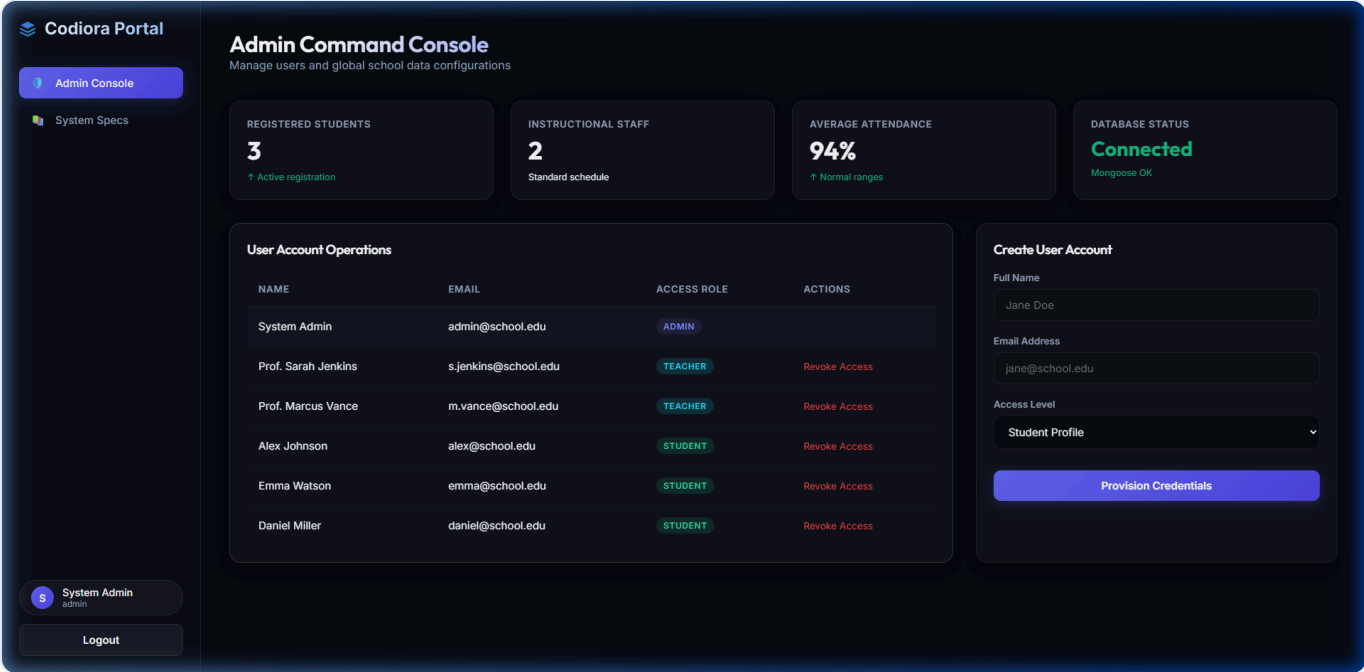
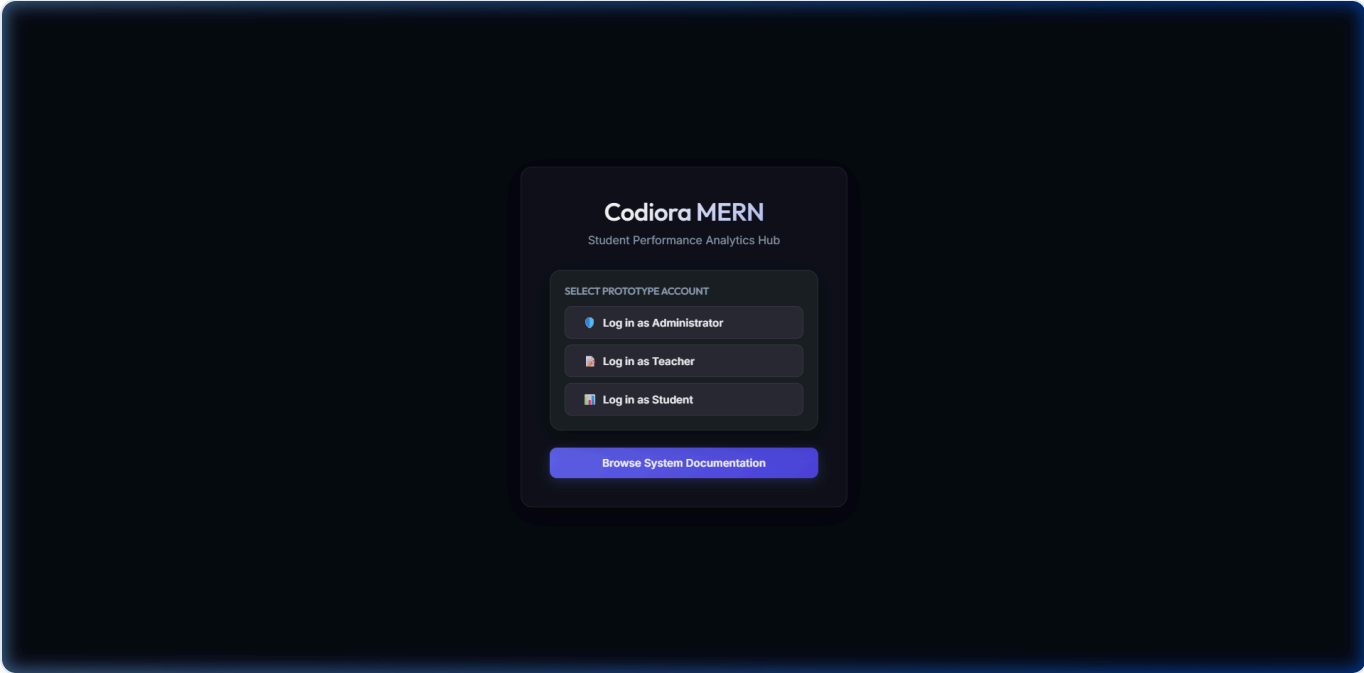
- Project initialized as Git repository (`git init`).
 - Formulated `.gitignore` exclusions.
 - Recorded first commit representing initial code structures and blueprints.
-

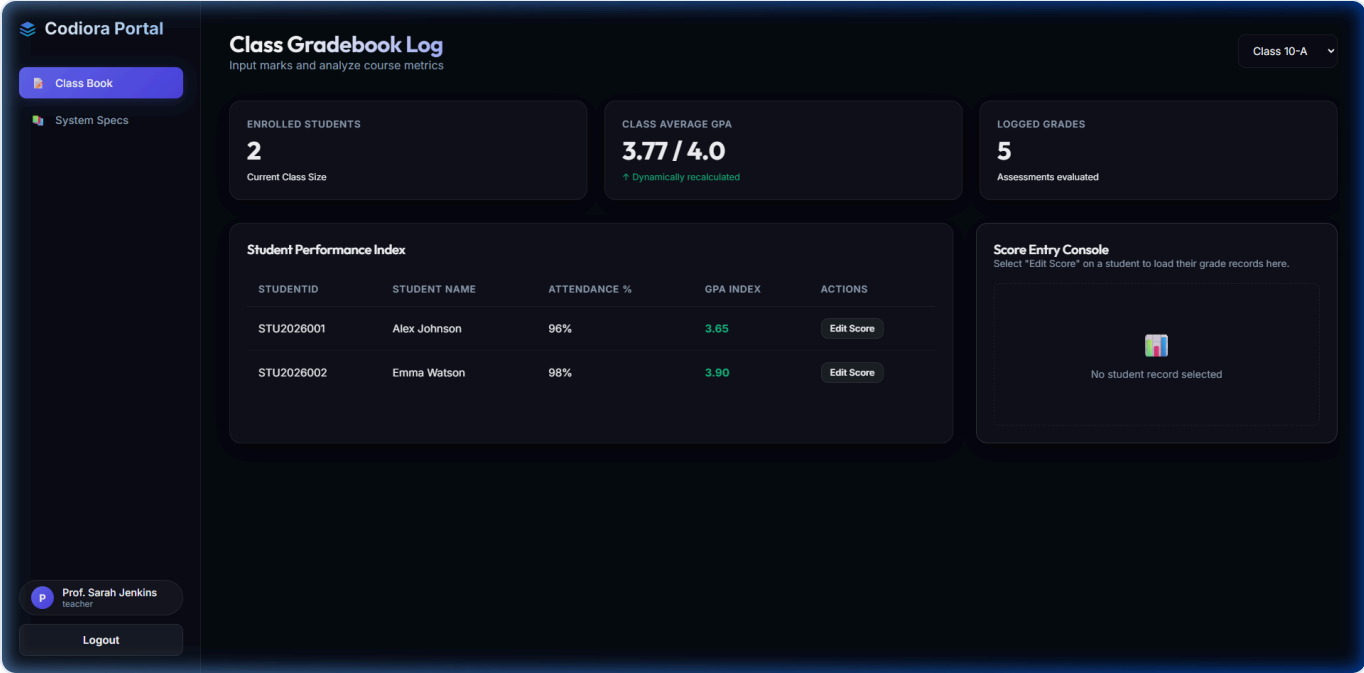
Validation Results

We verified the frontend application using a headless browser subagent to execute user actions:

- **Login screen:** Checked loading states and role selectors.
- **Admin Dashboard:** Confirmed metrics grids and account management tables render perfectly.
- **Teacher Dashboard:** Selected class dropdown, loaded grades, and modified a student score. Verified that class average GPAs dynamically recalculate.

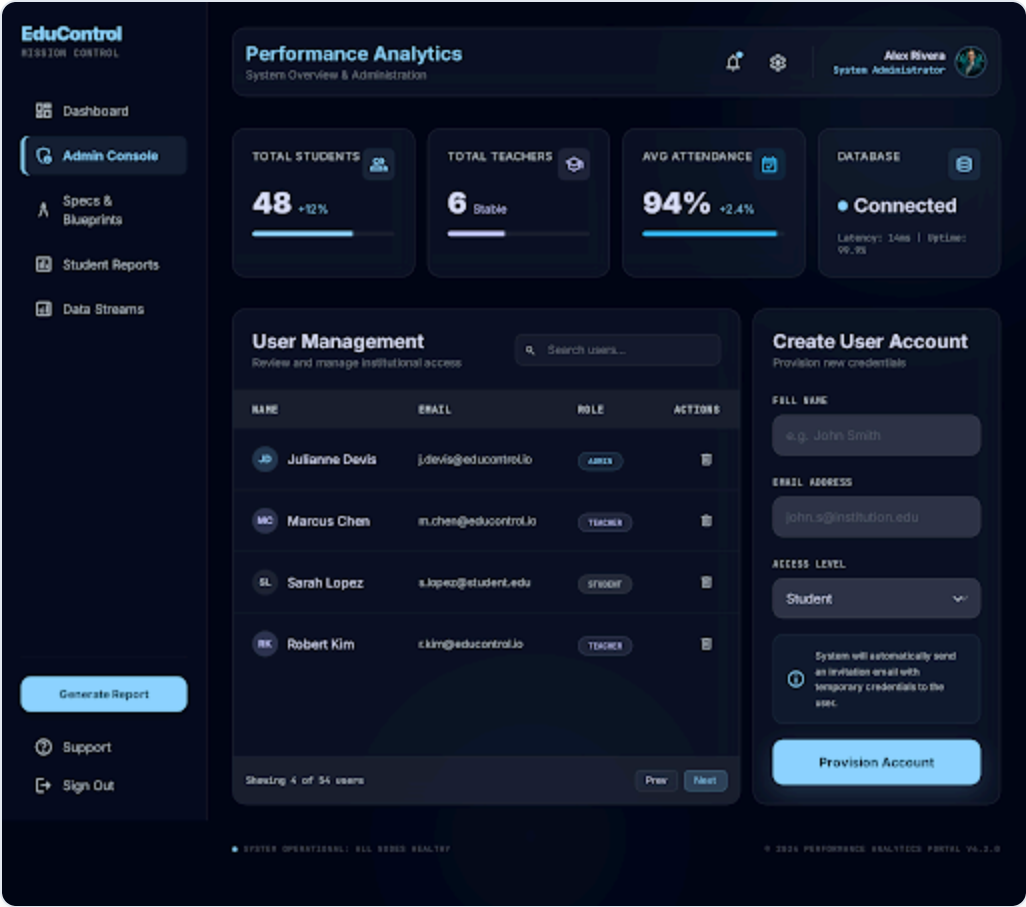
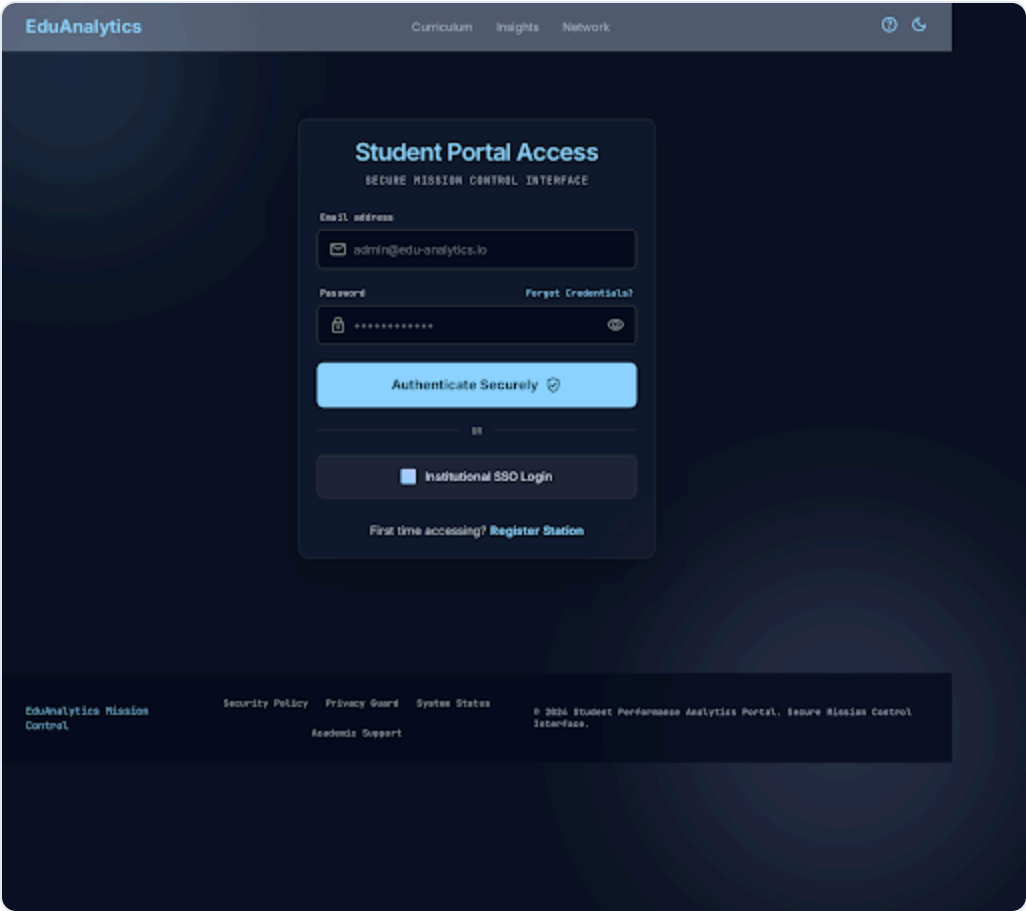
Here are the visual wireframe screenshots captured from the live web prototype:

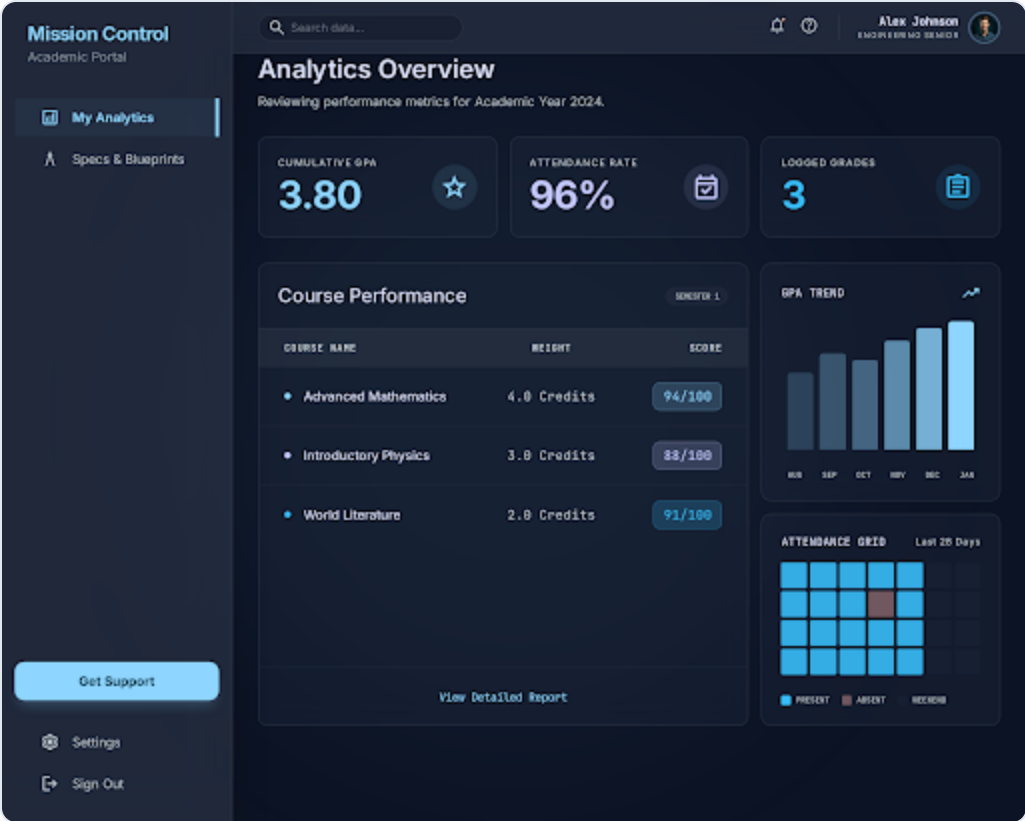
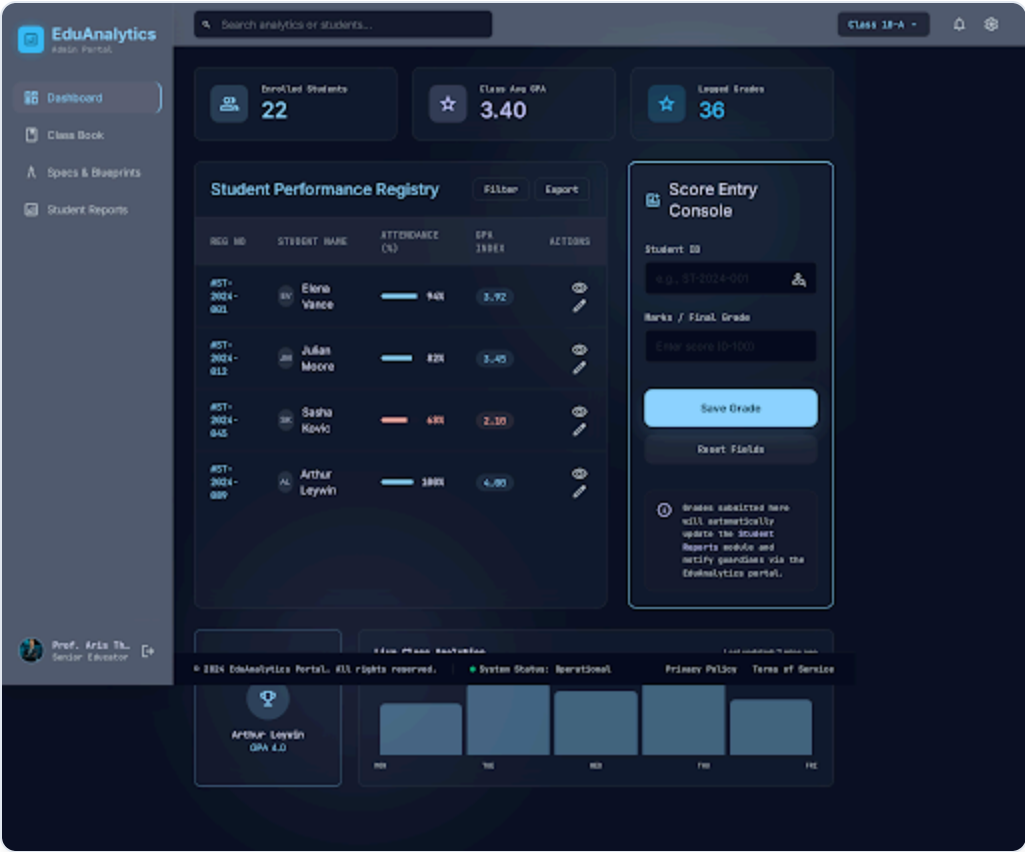




Stitch / Figma Wireframe Deliverables

We have initialized the visual wireframe designs inside **Stitch** (our Figma-integrated design environment) under the project resource ID: `projects/3602507402818866855` . Below are the high-fidelity screens designed using the glassmorphism theme tokens:





- Completed a production build (`npm run build`) on the React web client to confirm zero syntax errors.

- Verified file listings and repository states.